

复习台 · 今天要过哪几张

今天 9 月 23 日 (周三) 起算日 2026.09.23 节奏 1 · 3 · 7 · 30 天

打开就一个动作：过完今天这几张，然后关掉。日期是算出来的，不会过期变成废纸；逾期没过的会红字跳出来——上一版就是烂在这两点上（放了 9 天没人管，第 2 课的 404 三步排查法 17 天后彻底忘光）。

你在这里 > 阶段 A > ② 编程主线 > 复习 (跨全部课)

打开全局思维导图 >

日历 完整的 1-3-7-30 日历

同一张卡会被问 4 次——间隔越来越长，这是在跟遗忘曲线对着干。每格显示"过了几张 / 共几张"。

哪一组的卡	1 天 · 9.24 周四	3 天 · 9.26 周六	7 天 · 9.30 周三	30 天 · 10.23 周五
第 1 课 · 符号与语法 原学 2026.09.06 · 4 张	0 / 4	0 / 4	0 / 4	0 / 4
第 2 课 · 工程习惯 原学 2026.09.06 · 3 张	0 / 3	0 / 3	0 / 3	0 / 3
第 3 课 · 数据模型与 JOIN 原学 2026.09.23 · 4 张	0 / 4	0 / 4	0 / 4	0 / 4

起算日 2026.09.23。第 1、2 课的卡原本学于 2026.09.06，它们的 1/3/7 三档全部逾期且从未做过——09.23 第 3 课当场重测，证实已经忘光。⇒ **重学即重新起算**（今晚重新过了这几张），旧账不勾销、但也不再挂着吓人。

不会的怎么办

点「没过，再来」——那张卡会重新回到待过状态，下次打开还会出现。不许因为"已经过过一次"就把它当过了。1-3-7-30 的意思不是"读过 4 遍"，是"4 次都能自己说出来"。

抽查 顺手带走：JOIN 的诊断三步

- 单独数绳子表：`SELECT * FROM prescription_ingredients;`（实测 49 行）
- 再数 JOIN 的结果行数
- 两数一致 ≠ 配对正确——还要挑一行人工核对内容（id 重叠时会"数字对、内容全错"）
- 数字明显对不上（空 / 多很多）→ 几乎一定是 `ON` 写错

这是第 3 课最实用的迁移技能——下次 JOIN 查出来不对，从这三步走。

全部 全部问答 (打印用)

共 11 张。用手盖住答案那一栏自测 —— 能自己说出来才算过。

第 1 课 · 符号与语法 (原学 2026.09.06)

1. `=` 和 `==` 有什么区别?

`=` 是动作, `==` 是问题。更本质的区别是**有没有副作用**: `a = 5` 之后 `a` 变了; `a == 5` 吐出 `True`, 世界一点没变。写错 Python 会拦你: `if x = 5:` 直接 `SyntaxError`。

2. 装饰器的 `@` 到底干了什么?

迎宾员登记本: 进门先登记, 再放你进去。它不改函数体, 只是在外面包一层 —— 所以 `@app.get("/x")` 是"先登记这个路径, 再执行下面的函数"。

3. 推导式里的 `if` 该放哪?

放 `for` 后面 (那是筛选): `[x for x in xs if x > 0]`。只有带 `else` 的才是三元表达式, 那时才写前面。

4. `*` 和 `**` 怎么分?

一星乘、二星方 (`*` 乘、`**` 乘方); 在参数里则是 `*args` 收位置、`kwargs` 收关键字。

第 2 课 · 工程习惯 (原学 2026.09.06)

1. 浏览器报 404, 你的三步排查法是什么?

菜单 → 路径 → 端口: ① 先开 `/docs` 看这个接口**在不在菜单上** (不在 = 你记错了接口名) ② 你请求的 URL 和代码里 `@app.get("/xxx")` **一字不差**吗 ③ 两个端口 (如 8899 / 8000) 是不是**同一个服务**。

2. `commit()` 和 `close()` 分别管什么？忘了哪个才真出事？

`commit()` = 下单（确定落盘）； `close()` = 托盘放回（用完收摊）。

实测（Python 3.13.9 / SQLite 3.51.0）：

- `commit`  / `close`  → 另一个连接读到 **1 行**（数据在！）
- `commit`  / `close`  → **0 行**（丢了）

⇒ 丢数据的是 `commit`，不是 `close`；`close` 忘写的代价是占资源（未关的连接会锁住文件，实测删文件报 WinError 32）。

3. Pydantic 的 `BaseModel` 是干什么的？数据不合法会怎样？

声明式校验 —— 你在类里写字段和类型，它自动检查。不合法返回 **422**（不是 500）：422 = 你传的东西不对，500 = 我的代码崩了。

第 3 课 · 数据模型与 JOIN（原学 2026.09.23）

1. 什么是“绳子表”？为什么多对多必须要第三张表？

绳子的每一行 = 一条关系，不是一个人。

小明同时在 1 群和 2 群 → 他在绳子表里占**两行**。 `joined`（什么时候进的）、`role`（在这个群里是管理员还是成员）也都是**每条绳子各自带一个值** —— 放人表里放不下（一个人加 10 个群要存 10 行），放群表里也放不下（群有 200 人要把群名存 200 遍）。

2. `ON` 和 `WHERE` 有什么区别？

`ON` = 怎么配（配对规则）； `WHERE` = 要哪些（筛选）。

实测：三表 JOIN 不写 `WHERE` → 出 **3 行**（所有群的成员都算上）；补上 `WHERE groups.id = 1` → **2 行**。差的就是这一句。

3. `SELECT persons.name FROM persons JOIN group_members ON persons.id = group_members.uid WHERE group_members.gid = 1;` 会出几行？为什么？

2 行。行数 = 配对成功的次数，不是任何一张表里的行数。

1 号群有 2 个成员 → 配对成功 2 次 → 2 行。

4. 找错： `JOIN group_members ON groups.id = group_members.uid` 错在哪？会怎样？

群 id 去对人 id —— 绳子两头接反了。

两种后果，都不报错：

- id 空间**不重叠**（人 7/9、群 1/2）→ **0 行**，空的，一眼看得出来
- id 空间**重叠**（两套都从 1 开始编号 —— 你的 `prescriptions.db` 就是）→ **照样出数据，而且看着挺合理** ● 这才是最危险的

⇒ 所以 **JOIN 要核对的从来不是语法，是配对对不对**；"不为空"绝不等于"对"。

复习台 · 2026.09.23 建（T-24 / P-015 的修复件） | 数据源 `_全局进度.js` 的 `LabProgress.review`

按 **Ctrl+P** 可直接打成 A4 贴墙上（打印时按钮会自动隐藏） | 主题 `_主题.css`